

C-SQD MVP Build Plan

1 Purpose

The purpose of the C-SQD MVP is to demonstrate that scholarly evaluation can be made structured, compensable, discoverable, and reusable.

The MVP should not attempt to replace journals, host copyrighted articles, or build a general-purpose literature platform. Its initial goal is narrower: create a review layer over reputable open-access scholarly objects and prove that reviewers can produce typed, useful evaluations through a paid workflow.

The core product loop is simple. C-SQD identifies a reviewable scholarly object, routes it to a reviewer, captures structured evaluation, publishes the resulting review object, and records the associated payment or bounty state.

2 Platform Principle

C-SQD should be a review platform rather than a paper repository.

For each paper, the platform stores metadata and canonical access locations, not article PDFs. This avoids unnecessary copyright risk and keeps the platform focused on its primary asset: structured evaluation.

The stored scholarly object should include identifiers, title, authors, journal, publication date, abstract when permitted, license metadata, canonical publisher URL, open-access landing-page URL, full-text URL if available, PDF URL if available, and metadata provenance.

The article remains with the publisher or open repository. The review graph lives in C-SQD.

3 Initial Literature Scope

The MVP should begin with a curated subset of reputable open-access journals and open-access articles from reputable mixed-access journals.

The first corpus should be large enough to support reviewer choice but small enough to keep ingestion, metadata quality control, and product testing manageable.

A reasonable initial scope is biomedical and life-science literature, including open-access articles from major journal families, selected fully open journals, and public repositories. The system should prioritize articles with reliable DOI metadata, clear license metadata, and stable canonical URLs.

4 Core Data Model

The central object is the scholarly object. It represents the work being evaluated, independent of where the work is hosted.

The MVP data model should include the following entities:

- User
- ReviewerProfile
- Journal
- ScholarlyObject
- ExternalArticleLocation
- ReviewAssignment
- EvaluationFact
- ReviewEpisode
- SynthesisReview
- ErrorClaim
- Bounty
- PaymentRecord
- AuditEvent

The important conceptual move is to adapt the Fact–Episode–Narrative structure to scholarly evaluation.

An **EvaluationFact** is an atomic evaluative assertion. A **ReviewEpisode** groups related facts into a coherent review event, such as an **ElementReview**, bug bounty submission, author response, or adjudication. A **SynthesisReview** provides narrative interpretation across facts and episodes.

This structure allows peer review to become computable without eliminating prose.

5 Backend Architecture

The backend should be written in Rust.

Rust is appropriate because C-SQD requires explicit state transitions, strong typing, auditability, payment-adjacent logic, and careful permission control. The backend should own the review state machine, assignment state, publication state, bounty state, and payment state.

The initial stack should be conventional and reliable: Rust API server, PostgreSQL, object storage only for user-generated attachments, a search index for scholarly objects and reviews, and an external payment provider.

WebAssembly should be introduced selectively, not as the core backend model. Shared Rust validation logic can later be compiled to WebAssembly for browser-side validation of **ElementReviews**, but the first priority should be a stable server-side platform.

6 Paper Ingestion

The ingestion layer should retrieve and normalize metadata from trusted scholarly metadata sources.

The ingestion process should create or update **ScholarlyObject** records, attach external article locations, record license and provenance information, and mark whether native C-SQD article display is permitted.

The MVP does not need perfect full-text parsing. It needs reliable metadata, stable links, open-access status, journal/source classification, and enough article information for reviewers to select and evaluate work.

Each ingestion event should be auditable. The platform should know where each metadata field came from and when it was last refreshed.

7 Reviewer Flow

The reviewer flow is the core MVP experience.

A reviewer creates an account, completes a reviewer profile, selects expertise areas, and becomes eligible for assignments. A review assignment identifies the scholarly object, review type, compensation status, deadline, conflict-of-interest requirements, and publication expectations.

The reviewer then enters a review workspace. The workspace displays article metadata, source links, review instructions, and structured review forms. If the article can be shown natively, the workspace may include a native article view. Otherwise, it should provide the canonical article link and keep the review editor in C-SQD.

The reviewer submits the review, the system validates required fields, and the review enters a submitted state. Depending on the workflow, it may be published immediately, routed for administrative quality control, or held for author response.

8 ElementReviews

ElementReviews are the distinctive unit of C-SQD.

An ElementReview evaluates a specific aspect of a scholarly object. The first MVP should support a small number of review elements rather than a large taxonomy.

The initial elements should include claim support, methodological adequacy, statistical adequacy, data availability, code availability, interpretation strength, and reproducibility concerns.

Each ElementReview should generate one or more **EvaluationFact** records. These records should be typed, severity-scored where appropriate, connected to the scholarly object, and optionally linked to article sections, figures, tables, claims, or external evidence.

9 SynthesisReviews

A SynthesisReview is a structured narrative assessment of the scholarly object.

The MVP SynthesisReview should summarize the contribution, major strengths, major weaknesses, reliability concerns, and overall evaluation. It should be able to cite or incorporate relevant ElementReviews.

The purpose of the SynthesisReview is not to replace ElementReviews but to make their implications legible.

10 Bug Bounty Infrastructure

The bug bounty system should be implemented as a specialized review workflow.

A bug bounty begins with an `ErrorClaim`. The claim identifies the scholarly object, the alleged error, the affected part of the work, the evidence supporting the claim, the proposed severity, and the requested bounty category.

The claim then enters a triage state. It may be rejected, accepted for evaluation, routed to expert reviewers, sent to the author for response, validated, invalidated, or resolved as partially supported.

The MVP does not need automated bounty adjudication. It needs a clean state machine, public or private disclosure controls, evidence attachment, reviewer assignment, and payment tracking.

Bug bounties should be treated as part of the same evaluation graph as ordinary reviews. A validated error claim is not merely a support ticket. It is a structured contribution to the scholarly record.

11 Payments

The MVP should support paid review tracking even if early payment operations are partly manual.

Each paid review should be associated with a payer, reviewer, amount, completion condition, approval state, and payout state. Each bounty should similarly track funding source, reward amount, validation condition, and payout status.

The payment layer should not determine scientific validity. It records the economic state associated with review and bounty workflows.

12 Administrative Workflow

The MVP requires a lightweight administrative interface.

Administrators should be able to manage journals, scholarly objects, reviewer profiles, assignments, submitted reviews, bug bounty claims, payments, and publication states.

This interface is important because the early platform will require manual judgment. The system should not pretend to automate reviewer quality, conflict resolution, or scientific adjudication before sufficient data exists.

13 MVP Milestones

The first milestone is the metadata and scholarly object layer. C-SQD should ingest a curated corpus of reputable open-access articles and expose searchable article records with canonical source links.

The second milestone is the reviewer account and assignment workflow. A reviewer should be able to receive, accept, complete, and submit a structured review assignment.

The third milestone is the `ElementReview` editor. Structured evaluations should be captured as typed `EvaluationFacts`.

The fourth milestone is `SynthesisReview` support. The platform should support article-level narrative evaluation connected to structured facts.

The fifth milestone is bug bounty infrastructure. Users should be able to submit error claims, route them through triage, and track validation and reward status.

The sixth milestone is payment tracking. Paid reviews and bounties should have clear compensation states, even if early payouts require administrative approval.

14 What to Exclude From the MVP

The MVP should not include Observatory, a desktop application, broad publisher coverage, local storage of article PDFs, automated scientific judgment, complex reputation algorithms, or full-text ingestion at scale.

These may become important later, but they are not required to prove the initial thesis.

The MVP should prove that C-SQD can create a trusted, structured, paid review layer over existing scholarly literature.

15 Conclusion

C-SQD should begin as a web-native review platform over externally hosted scholarly objects.

The platform stores metadata, canonical access locations, structured reviews, error claims, bounties, payments, and audit history. It does not need to store article PDFs to create value.

The deeper architectural advantage is the adaptation of Fact–Episode–Narrative structure to scholarly evaluation. EvaluationFacts make review claims computable. ReviewEpisodes organize evaluative activity. SynthesisReviews make the resulting state intelligible.

The MVP succeeds if it demonstrates that this structure can support real reviewers, real papers, real compensation, and real scientific correction.

16 Browser Extension Companion

The MVP should include an optional cross-browser browser extension as a first-class reviewer companion. The core C-SQD web app must remain browser-agnostic and fully usable without the extension. The extension exists to improve review workflows for externally hosted journal and publisher content while preserving the rights-aware platform model.

The extension should connect a publisher or repository page to the C-SQD review graph. It should detect a DOI, canonical URL, title, and basic article metadata from the current page; match the page to an existing C-SQD scholarly object or start a create/import flow; and open a C-SQD companion panel or sidebar for drafting ElementReview facts while the reviewer reads the externally hosted work.

Where a browser-native side panel is available, the extension should use that model so the reviewer can keep the publisher article in the main browser content area and work in C-SQD alongside it. Where side panels are unavailable or unreliable, the extension should fall back to opening a separate-window or side-by-side C-SQD workspace.

Implementation should use the cross-browser WebExtensions model where practical. Chrome and Edge may use side-panel support, Firefox may use sidebar support, and Safari support may require Safari Web Extension packaging or conversion.

The extension must not bypass publisher controls, scrape paywalled article text, store unauthorized publisher content, or make C-SQD responsible for hosting externally controlled works. Its purpose is to connect externally hosted scholarly content to C-SQD review activity, not to copy or replace the publisher article experience.